Veri Yapıları Lab 4

Özyineleme (Recursion)

Bir fonksiyonun problemi çözmek için kendisini daha küçük parametrelerle çağırmasıdır.

Her yinelemeli problem iki parçadan oluşur:

Temel Durum (Base Case): Özyinelemeyi durduran ve doğrudan sonuç döndüren sınır koşulu.

Genel Durum (General Case): Problemi küçülterek fonksiyonun kendini tekrar çağırdığı kısım.

Çalışma Mantığı & Çağrı Yığını (Call Stack):

- Fonksiyon her çağrıldığında bellekte (Stack) yerel değişkenleri ve dönüş adresini saklayan yeni bir alan (stack frame) açılır.
- Base case'e kadar yığına ekleme yapılır, ardından sonuçlar geriye doğru hesaplanarak toplanır.

Sonsuz Özyineleme & Sınırlar:

- Temel duruma ulaşılamazsa veya yanlış kurgulanırsa program Stack Overflow (Yığın Taşması) hatasıyla çöker.
- Özyineleme kodun okunabilirliğini artırsa da çağrı yığını nedeniyle döngülere (iteration) kıyasla ek bellek ve zaman maliyeti getirir.

Örnek 1: Sonsuz Özyineleme ve Yığın Taşması (Stack Overflow)

Bu örnek, bir fonksiyona durma koşulu (base case) konulmadığında belleğin yığın (stack) bölgesinin nasıl dolduğunu ve programın işletim sistemi tarafından neden zorla sonlandırıldığını göstermek için kullanılır.

```c
#include <stdio.h>

void testStackOverflow(int derinlik) {
    printf("Cagri derinligi: %d\n", derinlik);
    // Base case bilerek konulmaz; yigin dolana kadar devam eder
    testStackOverflow(derinlik + 1);
}

int main(void) {
    testStackOverflow(1);
    return 0;
}
```

Örnek 2: Faktöriyel Hesabı (İteratif vs. Rekürsif Karşılaştırması)

Faktöriyel tanımı (n! = n \times (n-1)!), bir problemi kendi alt parçalarına bölme mantığını anlatmak için temel örnektir. Burada aynı işlemin döngü (iteration) ve kendini çağırma (recursion) yöntemleriyle yazımı karşılaştırılır.

```c
#include <stdio.h>

// Ozyinelemeli Cozum
long long factorialRec(int n) {
    if (n <= 1) // Base case (0! = 1, 1! = 1)
        return 1;
    return n * factorialRec(n - 1); // General case
}

// Iteratif Cozum
long long factorialIter(int n) {
    long long sonuc = 1;
    for (int i = 1; i <= n; i++) {
        sonuc *= i;
    }
    return sonuc;
}

int main(void) {
    int sayi = 5;
    printf("%d! (Rekursif): %lld\n", sayi, factorialRec(sayi));
    printf("%d! (Iteratif): %lld\n", sayi, factorialIter(sayi));
    return 0;
}
```

Örnek 3: Tek Yönlü Bağlı Listeyi Tersten Yazdırma

Özyinelemede çağrı anı ile geri dönüş (unwinding) anı arasındaki farkı gösterir. Ekrana yazdırma komutu özyinelemeli çağrının sonrasına yazıldığı için liste sonuna gidilir, ardından fonksiyonlar yığından boşaltılırken elemanlar tersten basılır.

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

// Bagli listeyi tersten yazdirma
void printReverse(struct Node* head) {
    if (head == NULL) // Base case
        return;

    // Once listenin sonuna kadar git
    printReverse(head->next);

    // Geri donus yolunda (stack bosalirken) degeri yazdir
    printf("%d ", head->data);
}

void push(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = (*head_ref);
    (*head_ref) = new_node;
}
```

```c
int main(void) {
    struct Node* head = NULL;
    // Liste: 6 -> 10 -> 14 -> 20
    push(&head, 20);
    push(&head, 14);
    push(&head, 10);
    push(&head, 6);

    printf("Liste tersten: ");
    printReverse(head);
    printf("\n");
    return 0;
}
```

Alıştırma: Hanoi Kuleleri (Towers of Hanoi)

Problem ve Kurallar: Elimizde 3 adet kule (Kaynak: A, Yardımcı: B, Hedef: C) ve büyükten küçüğe dizilmiş n adet disk bulunmaktadır. Amacımız, tüm diskleri A kulesinden C kulesine taşımaktır.

- Kural 1: Her hamlede yalnızca tek bir disk taşınabilir.
- Kural 2: Hiçbir disk, kendisinden daha küçük bir diskin üzerine koyulamaz.
- Kural 3: Diskleri aktarmak için yalnızca bir yardımcı kule kullanılabilir.



Özyinelemeli (Recursive) Düşünme Mantığı: n adet diski hedefe ulaştırmak için problemi 3 temel adıma böleriz:

1. Alt Problem 1 (Genel Durum): En alttaki en büyük diski boşa çıkarmak için üstteki n-1 diski kaynak kuleden yardımcı kuleye taşı.
2. Doğrudan Çözüm (Temel Durum): Kalan 1 diski (en büyük diski) doğrudan kaynaktan hedef kuleye taşı.
3. Alt Problem 2 (Genel Durum): Yardımcı kulede bekleyen n-1 diski, hedef kuleye taşı.

Verilen disk sayısı n için tüm hamleleri ekrana sırasıyla yazdıran özyinelemeli bir C fonksiyonu tasarlayınız.

Alıştırma: Hanoi Kuleleri (Towers of Hanoi)

Fonksiyon imzası:

```
void hanoi(int n, char kaynak, char hedef, char yardimci);
```

Örnek Çalışma Çıktısı (n = 3 için):

```
Disk sayisini giriniz: 3

1. Diski A kulesinden C kulesine tasi
2. Diski A kulesinden B kulesine tasi
3. Diski C kulesinden B kulesine tasi
4. Diski A kulesinden C kulesine tasi
5. Diski B kulesinden A kulesine tasi
6. Diski B kulesinden C kulesine tasi
7. Diski A kulesinden C kulesine tasi

Toplam yapilan hamle sayisi: 7
```